

Spring Boot Annotations - Simple Guide for Beginners

What are Annotations?

Annotations are like special labels or tags that you put on your Java code to tell Spring Boot how to handle different parts of your application. Think of them as instructions for Spring Boot.

Basic Application Setup Annotations

1. @SpringBootApplication

- **What it does:** This is like the "start button" for your Spring Boot application
- **Where to use:** Put this on your main class (the one with the main method)
- **Why it's special:** This one annotation actually does the work of three annotations:
 - Finds and sets up everything automatically
 - Scans for components in your project
 - Marks the class as a configuration class

2. @Bean

- **What it does:** Tells Spring "Hey, this method creates an object that you should manage"
- **Where to use:** On methods that create objects
- **Simple example:** When you want Spring to handle creating and managing objects for you

3. @Component

- **What it does:** Labels a class as something Spring should manage
 - **Where to use:** On any class you want Spring to create and manage automatically
 - **Think of it as:** Telling Spring "This is an important class, please take care of it"
-

Dependency Injection (Getting Objects Automatically)

4. @Autowired

- **What it does:** Automatically gives you the object you need without creating it yourself
- **Where to use:** On variables, constructors, or setter methods
- **Like having:** A helpful assistant that hands you tools when you need them
- **Works with:** @Qualifier (when there are multiple options) and @Resource

5. @Qualifier

- **What it does:** Helps choose which specific object you want when there are multiple similar ones
- **Where to use:** Together with @Autowired
- **Example:** Like saying "I want the red car, not the blue car"

6. @Lazy

- **What it does:** Creates objects only when you actually need them, not at startup
 - **Where to use:** On classes or methods
 - **Benefit:** Saves memory and makes your app start faster
-

Layer-Specific Annotations (Organizing Your Code)

7. @Controller

- **What it does:** Marks a class that handles web requests and returns web pages
- **Where to use:** On classes that handle user requests from websites
- **Returns:** Usually HTML pages or views

8. @RestController

- **What it does:** Like @Controller but returns data directly (JSON/XML) instead of web pages
- **Where to use:** When building APIs that send data to mobile apps or other services
- **Modern choice:** Most new applications use this

9. @Service

- **What it does:** Marks classes that contain your business logic
- **Where to use:** On classes that do the main work of your application
- **Think of it as:** The "brain" of your application where decisions are made

10. @Repository

- **What it does:** Marks classes that talk to the database
 - **Where to use:** On classes that save, update, delete, or find data
 - **Think of it as:** The "librarian" that manages your data storage
-

Configuration and Properties

11. @Value

- **What it does:** Reads values from your configuration files
- **Where to use:** On variables where you want to inject configuration values
- **Example:** @Value("\${server.port}") gets the port number from settings

12. @PropertySource

- **What it does:** Tells Spring where to find your configuration files
- **Where to use:** On configuration classes

13. @ConfigurationProperties

- **What it does:** Groups related configuration properties together

- **Where to use:** On classes that represent configuration settings
- **Benefit:** Cleaner than using multiple @Value annotations

14. @Profile

- **What it does:** Makes certain configurations active only in specific environments
- **Where to use:** When you want different settings for development, testing, and production
- **Example:** Different database settings for testing vs production

15. @Scope

- **What it does:** Controls how many instances of an object Spring creates
 - **Options:**
 - singleton (default): One object for the whole application
 - prototype: New object every time you ask for it
 - request: New object for each web request
 - session: One object per user session
-

Web Request Handling

16. @RequestMapping

- **What it does:** Maps web URLs to your methods
- **Where to use:** On methods that handle web requests
- **Example:** Maps "/users" URL to your getUserList() method

17. @GetMapping

- **What it does:** Handles GET requests (like clicking a link or loading a page)
- **Where to use:** On methods that retrieve/display data

18. @PostMapping

- **What it does:** Handles POST requests (like submitting a form)
- **Where to use:** On methods that create new data

19. @PutMapping

- **What it does:** Handles PUT requests (updating existing data)
- **Where to use:** On methods that update complete records

20. @DeleteMapping

- **What it does:** Handles DELETE requests (removing data)
- **Where to use:** On methods that delete records

21. @RequestBody

- **What it does:** Converts JSON/XML from the request into Java objects
- **Where to use:** On method parameters when receiving data from clients

22. @PathVariable

- **What it does:** Extracts values from the URL path
- **Example:** In URL `"/users/123"`, extracts `"123"` as the user ID

23. @RequestParam

- **What it does:** Gets parameters from the URL query string
 - **Example:** In URL `"/search?name=John"`, extracts `"John"` as the name parameter
-

Error Handling

24. @ControllerAdvice

- **What it does:** Handles errors that happen anywhere in your application
- **Where to use:** On classes that deal with global error handling
- **Benefit:** One place to handle all errors instead of repeating code

25. @ExceptionHandler

- **What it does:** Handles specific types of errors and customizes the response
 - **Where to use:** On methods inside `@ControllerAdvice` classes
-

Database Related Annotations

26. @Entity

- **What it does:** Tells Spring this class represents a database table
- **Where to use:** On classes that map to database tables

27. @Table

- **What it does:** Specifies the exact table name in the database
- **Where to use:** On `@Entity` classes when the table name is different from class name

28. @Column

- **What it does:** Maps class fields to specific database column names
- **Where to use:** On fields when the column name differs from field name

29. @GeneratedValue

- **What it does:** Automatically generates values for database columns (like ID numbers)
- **Where to use:** On ID fields that should be auto-generated

30. @Transactional

- **What it does:** Ensures database operations are completed successfully or rolled back entirely
- **Where to use:** On methods that perform multiple database operations

31. @Temporal

- **What it does:** Specifies how date/time fields should be stored in the database
- **Where to use:** On date/time fields

32. @Transient

- **What it does:** Tells the database to ignore this field (don't save it)
 - **Where to use:** On fields that exist in your class but shouldn't be stored
-

Code Generation and Utilities

33. @Data (Lombok)

- **What it does:** Automatically creates getter, setter, toString, and other common methods
 - **Where to use:** On classes where you want to avoid writing repetitive code
 - **Benefit:** Keeps your code clean and short
-

Security and Access Control

34. @CrossOrigin

- **What it does:** Allows web browsers from other domains to access your API
- **Where to use:** On controllers when building APIs for web applications

35. @Secured

- **What it does:** Restricts access to methods based on user roles
- **Where to use:** On methods that should only be accessible to certain users

36. @PreAuthorize

- **What it does:** Checks permissions before allowing method execution
- **Where to use:** On methods that need complex access control

37. @PermitAll

- **What it does:** Allows everyone to access this method (no authentication required)
 - **Where to use:** On public methods like login or registration
-

Caching (Making Your App Faster)

38. @EnableCaching

- **What it does:** Turns on caching features for your application

- **Where to use:** On your main configuration class

39. @Cacheable

- **What it does:** Stores method results in memory to avoid repeated calculations
- **Where to use:** On methods that are called often with the same parameters

40. @CachePut

- **What it does:** Always executes the method and updates the cache with new results
- **Where to use:** On methods that modify data that's cached elsewhere

41. @CacheEvict

- **What it does:** Removes data from the cache
- **Where to use:** On methods that delete or invalidate cached data

Aspect-Oriented Programming (AOP) - Advanced Features

42. @Aspect

- **What it does:** Marks a class that contains cross-cutting concerns (like logging)
- **Where to use:** On classes that handle common functionality across your app

43. @Pointcut

- **What it does:** Defines where and when your aspect code should run
- **Where to use:** Inside @Aspect classes to specify method patterns

44. @Before

- **What it does:** Runs code before a method executes
- **Where to use:** For logging, security checks, or validation

45. @AfterReturning

- **What it does:** Runs code after a method completes successfully
- **Where to use:** For cleanup or success logging

46. @AfterThrowing

- **What it does:** Runs code when a method throws an exception
- **Where to use:** For error handling or logging failures

47. @Around

- **What it does:** Wraps around method execution, giving you control before and after
- **Where to use:** For timing, caching, or complex cross-cutting concerns

Quick Tips for Beginners

1. **Start with the basics:** Focus on `@SpringBootApplication`, `@RestController`, `@Service`, and `@Repository` first
2. **One step at a time:** Don't try to learn all annotations at once
3. **Practice with examples:** Create small projects to understand how each annotation works
4. **Read error messages:** They often tell you which annotations you might be missing
5. **Use IDE help:** Most IDEs will suggest appropriate annotations as you type

Remember: These annotations are tools to make your life easier. They handle a lot of complex setup automatically, so you can focus on writing your business logic!